Generating Company Names with a Character-Level Transformer

Mauro Reverberi

Udacity Institute of AI & Technology / Woolf

Master of Science in AI, Capstone Project 5: Generative AI Applications

November 6, 2026

Generating Company Names with a Character-Level Transformer

Overview

In this project I built a small generative model that writes new UK company names, one character at a time. It is a decoder-only Transformer that uses the GPT-2 classes of the Hugging Face Transformers library (Hugging Face, n.d.) without pretrained weights. I trained it from scratch in PyTorch on 80,142 names of real private limited companies on the Companies House register. My research question was whether such a model can generate plausible and varied company names. To answer it I counted repeated names to measure variety, read a random sample to judge plausibility, and checked how often a generated name copied a training name or had the same key as a company in the register. The task connects to my earlier capstone projects on company data, and the names could later serve as test inputs for a due diligence agent that matches company names.

Of the three options in the project (GAN, VAE and Transformer), I chose the Transformer. A company name is a short sequence of characters, and with self-attention the model can use all earlier characters to predict the next one (Vaswani et al., 2017). A GAN or a VAE would need a second network, and the GAN also a way to train through discrete characters (Goodfellow et al., 2016). The model reached a test loss of 1.1513 nats per character (the cross-entropy computed with the natural logarithm, lower is better) against 1.3819 for a character 5-gram model. The most important finding is that at temperature 0.5 the key of 44.25% of the names (885 of 2,000) matches a company in the snapshot under my own register key. Across all four temperatures there were 1,618 such matches. These are possible collisions with real companies, and only 16 of them are exact copies of training names.

Dataset or Prompt Description

Companies House publishes a free monthly snapshot of all live companies on the UK register (Companies House, n.d.). On October 16, 2026 I downloaded the snapshot of October 1, 2026 with 5,704,711 companies. Companies House does not limit how the information on the public register may be used, but possible rights of third parties in the data still apply (Companies House, 2019). The names are real register entries, nothing in the data is synthetic or generated, and I did not use this source in my earlier capstone projects.

My script prepare_dataset.py keeps only the private limited companies, so all names share one legal form. It removes names with characters outside an allowed set (6,984, for example names with accented letters), names longer than 60 characters (2,415), names that do not end with LTD, LTD. or LIMITED (7,971) and 11 repeated names. From the remaining 5,259,281 names it draws a random sample of 100,000 with seed 42.

If very similar names sit in training and test, the test partly measures what the model has memorized (Lee et al., 2022). The script therefore normalizes the names and puts names with the same first two words into one group, so ACME HOLDINGS LTD and ACME HOLDINGS (UK) LIMITED stay together. It splits the 95,078 groups about 80/10/10 into 80,142 training, 9,835 validation and 10,023 test names. The rule is simple and does not identify real company families, so some similar names can still land in different splits.

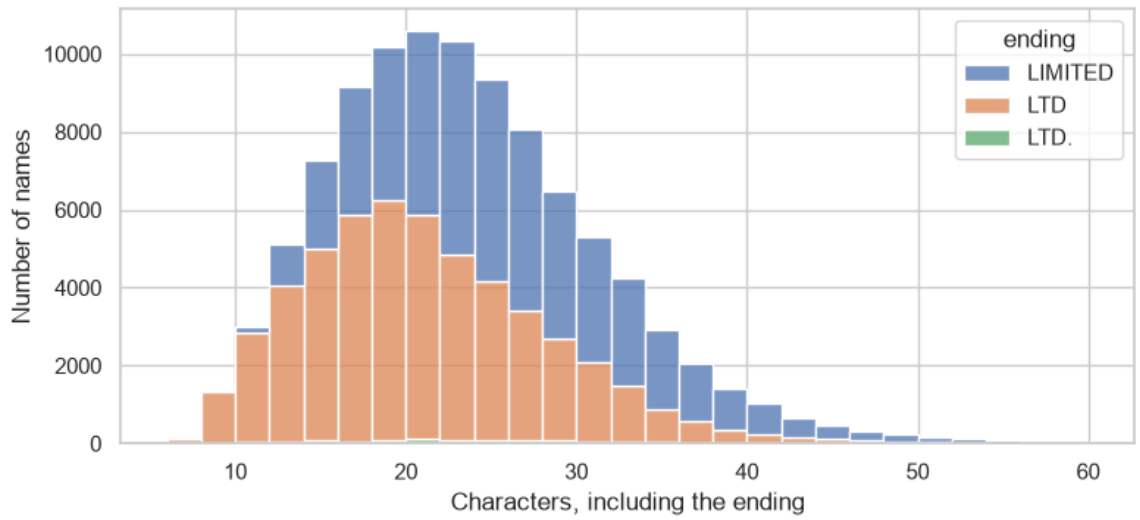
The script also computes a key for every live company, which I wrote myself. It follows the "same as" rules for company names, as Companies House (2026) describes them, so HANDS UK LTD and HAND'S LIMITED get the same key. It is not the official test. It does not apply every rule in the same order, for example ACME L.T.D. and ACME LTD get different keys, and it does not cover the exceptions or the separate "too like" test. The file register_key_hashes.npy

holds a 64-bit hash of the key of every company, 5,673,818 different keys in total. A key match is therefore only a possible collision, and a name without a match can still be the same as or too like a registered name.

The median name has 22 characters and 3 words, and Figure 1 shows that most names are between 10 and 40 characters long. In the sample 51,620 names end with LTD, 47,376 with LIMITED and 1,004 with LTD. (with a full stop). After the preparation 48 different characters remain. The most frequent words are generic business words like SERVICES (in 4.8% of the training names), PROPERTY (2.6%) and HOLDINGS (2.1%).

Figure 1

Length of the Company Names in Characters, by Ending



Model Design and Training Approach

I built the model with the Hugging Face Transformers library (Hugging Face, n.d.), which is part of my coursework and which I also used in Project 4. A GPT2Config sets the size, and GPT2LMHeadModel creates the model with random weights, so nothing is pretrained or downloaded. A character tokenizer maps the 48 characters of the training names to ids and adds a padding token and an end marker, which stands before and after every name. A token

embedding and a learned position embedding turn each character into a vector of 256 numbers. Four Transformer blocks follow, each with causal self-attention over eight heads and a feed-forward part from 256 to 1,024 to 256 units. As in GPT-2, each part has a layer norm in front and a residual connection around it (Radford et al., 2019). A final linear layer gives one logit per token and shares its weights with the token embedding, which is the main difference to the decoder-only Transformer from my course. The causal mask lets every position see only itself and the earlier characters, so the model learns to predict the next character (Vaswani et al., 2017). The model has 3,187,968 parameters. I chose this size as a practical starting point for training on my CPU. I did not compare smaller or larger models. In the notebook I also define the same architecture with my own PyTorch classes. I copied the weights of the library model into it and compared the logits of both on four validation names, and the largest difference was 1.2e-6.

I did not use the pretrained GPT-2 weights. GPT-2 was pretrained on about 8 million web documents (Radford et al., 2019), so I would not know which company names it had already seen. It also splits text into pieces of words, not single characters. Large language models can reproduce parts of their training data word for word (Carlini et al., 2021). With my own model I know which 80,142 names it saw during training and can check whether a generated name is one of them, but an exact match alone does not tell me how the model produced it.

The loss is the cross-entropy between the predicted distribution of the next token and the real next token, averaged over all characters and end markers, with the padding ignored. Cross-entropy fits this task because the model predicts a distribution over the next token (Goodfellow et al., 2016), and I compute it in my own training loop. The optimizer is AdamW (Loshchilov & Hutter, 2019) with batches of 128 names and the PyTorch default values, a learning rate of 1e-3

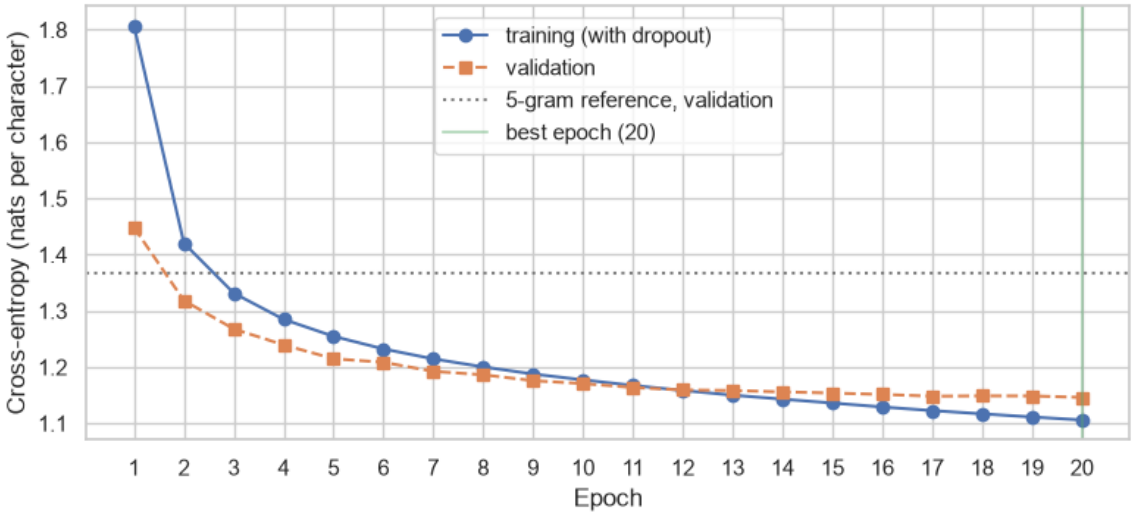
and weight decay of 0.01. I did not search for better values. Against overfitting I used dropout of 0.1 and, after 20 epochs, kept the weights of the epoch with the lowest validation loss, the selection idea of early stopping (Goodfellow et al., 2016). All random parts use seed 42.

As a reference without deep learning I built a character n-gram model. It counts which character follows the last n - 1 characters and smooths the counts with a small constant k. The best of 12 combinations was order 5 with k = 0.1, with a validation loss of 1.3680 nats per character. Figure 2 shows that the Transformer was below this value from the second epoch. Its validation loss reached 1.1707 in epoch 10 and then fell by only 0.0246 more to 1.1461 in epoch 20, while the training loss fell to 1.1059. I measured the training loss with dropout while the weights still changed, so its gap of about 0.04 to the validation loss is not a direct comparison. The validation loss did not show a sustained rise, so I see no clear sign of overfitting within 20 epochs. On the test split the Transformer reached 1.1513 nats per character, a perplexity of 3.16, against 1.3819 for the 5-gram model. This shows that it predicts held-out characters better than a model that only uses local contexts, but not which dependencies it learned.

I trained the model on the CPU of my Mac. One epoch took about 4 minutes and the whole training about 80 minutes. Repeated full runs gave the same loss in every epoch and the same generated names. On other hardware, for example with CUDA, the losses and generated names can differ. I did not test this.

Figure 2

Cross-Entropy per Character During Training



Output Evaluation and Interpretation

To generate names I use model.generate of the Hugging Face library with sampling. The model starts with the end marker and samples one character after the other until it samples the end marker again or reaches 60 characters. The temperature divides the logits before the softmax, so a low temperature makes likely characters more likely and a high temperature gives rare characters more chances. Sampling from the full distribution can produce strange choices from its unreliable tail, while choosing only the most likely tokens makes text bland and repetitive (Holtzman et al., 2020). I switched off top-k and top-p sampling, so only the temperature changes the distribution. I generated 2,000 names at each of the temperatures 0.5, 0.8, 1.0 and 1.2, plus 2,000 names from the 5-gram model at temperature 1.0.

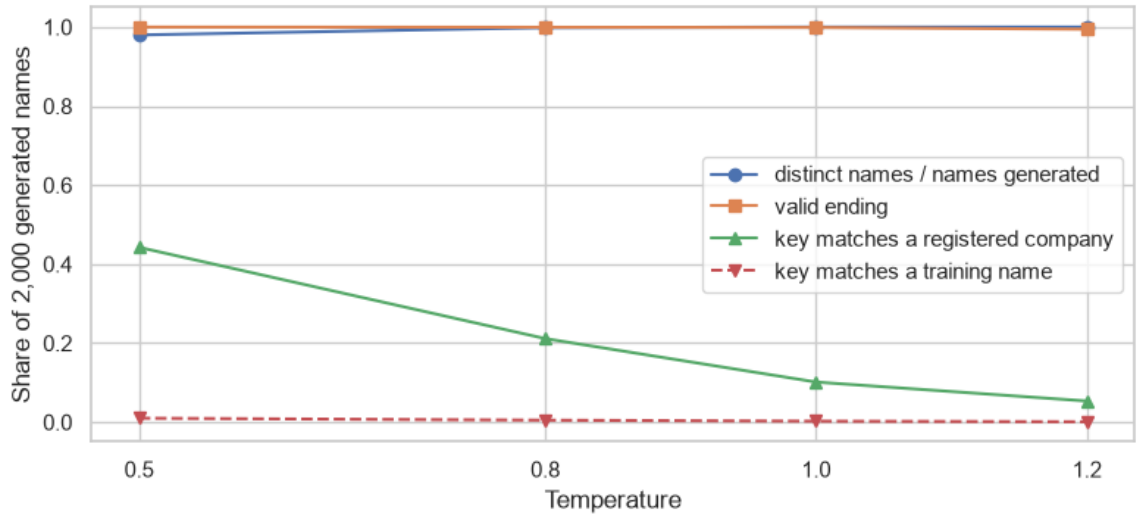
A low loss does not guarantee good samples, so generative models should be judged with checks that fit their use (Theis et al., 2016). I fixed my basic checks before I looked at the names and added the temperature rule and the filter check later. I counted the finished names, the names with a valid ending, the distinct share (different names divided by names generated) and the

exact copies of training names. I also checked whether the key of a name matches the key of a training name or of any company in the snapshot. All 10,023 real test names were found in the register, which tests the lookup but not how well my key follows the official rules.

The Transformer finished 7,993 of its 8,000 names, and 99.45% to 100% of its names had a valid ending. The distinct share was 98.00% at temperature 0.5 and 99.85% to 100% at the higher temperatures. Exact copies of complete training names were rare, 16 of 8,000. The register check showed a much bigger problem (Figure 3). At temperature 0.5 the key of 44.25% of the names matched a company in the snapshot, at 0.8 21.15%, at 1.0 10.15% and at 1.2 5.35%. Without matches with training keys these shares are at most 1.00 percentage point lower. Only 37 of the 1,618 matches had the key of a training name and 16 were exact copies, but my checks do not show whether the model reproduced parts of other names. Of 24 example matches in the notebook, many were short combinations of common words, like THE CARE COMPANY LIMITED, or short initials like PMGM LIMITED. At temperature 0.8, 47.58% of the names with two words matched, but only 10.14% of the names with four words.

Figure 3

Quality Checks and Key Matches by Sampling Temperature



To see how close the new names are to the training names, I represented each name with character 2- to 4-grams and used TF-IDF weighting (Salton & Buckley, 1988), computed with scikit-learn. I then measured its cosine similarity to every training name and kept the closest one. The check ignores word order and meaning, so it is not an identity test. Of all names, 19 reached a similarity of at least 0.999, 17 exact copies (16 from the Transformer and one from the 5-gram model) and twice ELITE CARE LIMITED against the training name CARE ELITE LIMITED. As a reference I used a random sample of 2,000 real validation names, with a median similarity of 0.584. The split keeps names with the same first two normalized words together, which can lower the similarity of the validation names to the training names. I used the validation names and not the test names, because I chose the temperature with this comparison. In Figure 4 the names at temperature 1.0 have almost the same distribution with a median of 0.575, while the names at 0.5 share more character groups with training names (median 0.685). The 5-gram names have a median of only 0.335. Six of its first ten names broke into random-looking characters like SIRN'YOP)@6WIR9, because an unseen context of four characters makes every token except padding equally likely under add-k smoothing, and only 32.60% of its names ended correctly. The Transformer produced much more regular names, although I did not test how much of this comes from attention alone.

Figure 4

Similarity of Each Name to Its Closest Training Name

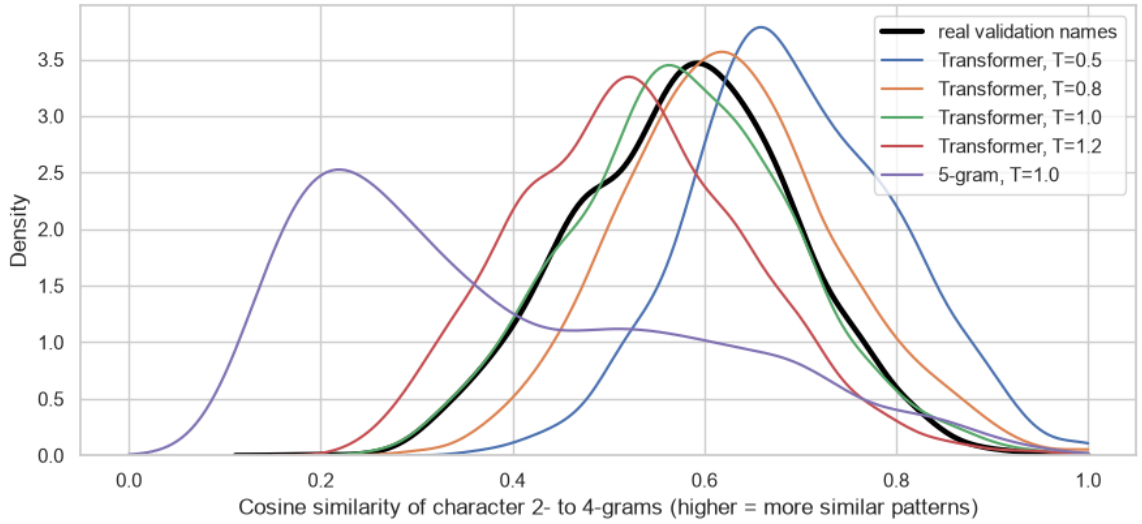
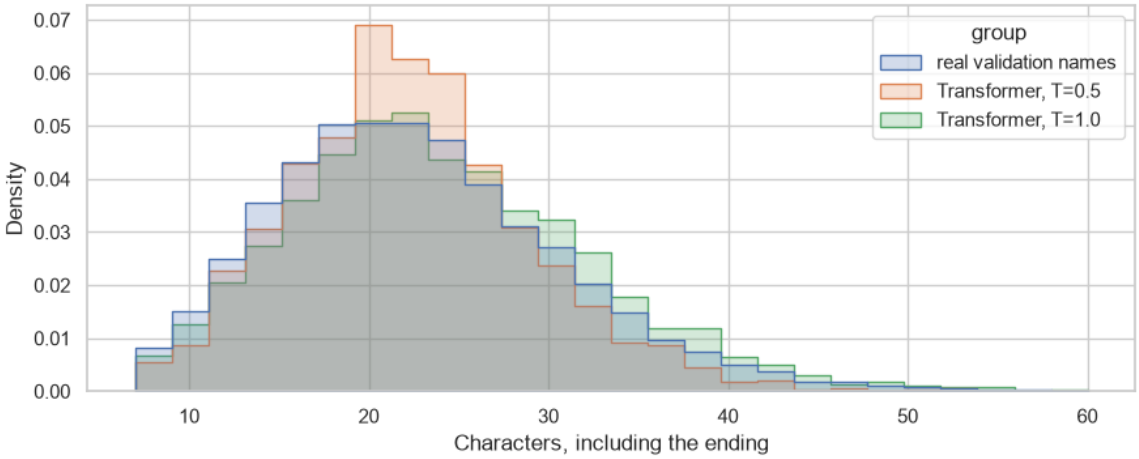


Figure 5 shows that the names at temperature 1.0 follow the real validation names in length up to about 26 characters but include more long names, with a mean of 24.18 characters against 22.95. I also counted the names with at least one word that is not in the training names, without the final LTD, LTD. or LIMITED. Such a word can be a real name, a code, a misspelling or an invented word, so this shows how new the words are and not how good the names are. The share is 51.10% for the real validation names, 62.85% at 1.0 and only 14.15% at 0.5. At 0.5 the model also overuses a few words, CONSULTING in 10.50% of the names instead of 1.57% in the training data and PROPERTIES in 7.05% instead of 2.18%. This fits the observation that decoding with only the most likely tokens leads to bland and repetitive text (Holtzman et al., 2020), although at 0.5 my model still samples from the full distribution.

Figure 5

Length of Generated and Real Names



Reading the names supports these numbers. I call a name plausible if I can read its words or name parts, if it has a typical structure and the right ending and if its brackets are closed. Of the 2,000 names at temperature 1.0, 1,795 pass my ending, duplicate and register-key checks. They are only candidates, because these filters do not check readability or brackets, and six of them have unbalanced brackets, like HOWLAND ESTATES (26 LIMITED. In a random sample of 15 candidates many look plausible to me, for example ZNM PLUMBING & HEATING SERVICES LIMITED or INDIAN CRICKET KITCHEN LIMITED. Others are weaker. 16228 OPERATIONS RVINE SERVICES LIMITED has a word that looks broken to me, and CAXPY CAPITAL INVESTMENT LIMITED starts with a word that is hard to read. Of the 8,000 names 14 failed my ending check, 11 of them at 1.2. Seven were cut off at the limit of 60 characters, like ALDERIS BUILDING SOLUTIONS & PROCESS ENGINEERING SERVICES LT, and others misspell the ending, like THE WAIFESAFE LLTD. At temperature 0.5 the opposite problem appears, CONSTRUCTION LIMITED, ARCHITECTURE LIMITED and ARCHITECTURE LTD each came out 4 times among 2,000 names.

I chose the temperature for the detailed examples with a rule on the validation names. It takes the temperature whose similarity values are closest to those of the real validation names. For each similarity value I compare the share of generated names and the share of validation names at or below this value, and the largest difference between the two shares is the distance. It is the statistic of the two-sample Kolmogorov-Smirnov test (SciPy, n.d.). I use it only to compare the two samples, not as a significance test. The rule picked 1.0, where the two shares never differ by more than 3.2 percentage points (a distance of 0.0320), against 0.1460 at 0.8, 0.2190 at 1.2 and 0.3820 at 0.5. On the length of the names 0.8 was closer than 1.0 (0.0435 against 0.0650). The rule looks at all generated names. After the filters, 1,795 names remain at temperature 1.0, and their distance rises from 0.0320 to 0.0674. This is still lower than 0.1044 at 0.8, but the gap is smaller, and I added this check only after the rule. This is an exploratory choice, because I wrote the rule after an earlier version of my notebook had compared the temperatures on the test names. A similar distance also says nothing about plausibility, and 10.15% of the names at 1.0 still match the register, so a register check is needed before any name is used.

Ethical Considerations and Responsible Use

The first concern comes directly from my results. A generated name can have the same key as a real company even if its key matches no training name. At temperature 1.0 this happened for 9.90% of the names and at 0.5 for 43.25%. These are possible collisions, not proven company identities. If a test or demo output is shared, a real company could appear in a context that has nothing to do with it, for example in a list of suspicious entities. Companies House generally rejects new names that are the same as an existing one (Companies House, 2026), and I used my own key, which is based on these rules, as a filter. Responsible use of this generator therefore includes the register check as a fixed step. Even then the check is not a

guarantee, because it covers only the UK register of one month and not the "too like" test. A name without a match can still be close to a UK company, belong to a company in another country or touch trademark rights. So the question of ownership here is less about copyright and more about confusion with a real company.

The second concern is personal names. Some generated names look as if they contain a person's name, for example HUGHES HOLDINGS LIMITED and PATRICK CONSULTANCY LIMITED, whose keys match registered companies. I did not verify the people behind them, but such a name could still be read as a statement about a person. Language models can reproduce personal information from their training data (Carlini et al., 2021). Exact copies were rare in my samples, 16 of 8,000, but that does not show that the model memorized nothing. So I would not publish generated names without the register check, and I would label them as generated test names that may overlap with real company names. The examples in the notebook are raw outputs and not approved test names.

The third concern is misuse. A generator of plausible company names could help someone to create names for shell companies or to imitate a real company in phishing. The model produces only names, and I did not measure whether it makes misuse easier than choosing names by hand. Anyone with the code and the data can train it again, and it produces thousands of names quickly, so I would not offer it as a public service. In my capstone it is meant only to create test inputs for a matching system, to check whether the system links a plausible name to a real company by mistake. There is also a bias. I removed names with accented letters and names with other endings, for example the Welsh form CYF, and the training names reflect which companies exist in the UK. A matching system tested only with such names would be tested less

on names from other languages, and I expect that matching errors matter there in compliance checks, although I did not test this.

Limitations and Future Improvements

My register key does not follow the official test in every detail, and I did not measure how often it merges or separates names differently from Companies House. I trained one configuration and always used seed 42, so I did not test how much the results change with another seed or another size of the model. All results come from the CPU of my Mac, and I could not test CUDA. I judged the quality of the names by reading samples myself, which is my own judgment and not a tested measure of plausibility. The test loss and the name-quality comparisons cover the selected private limited companies of one snapshot. The register lookup covers all live companies in that snapshot, but no other month or country.

Realistic next steps follow from these limits. First, the register check could add a fuzzy match for names that are only too similar, which is the same kind of matching the due diligence agent will need. Second, the evaluation could add a small blind test where several people try to tell real and generated names apart, and a rule for choosing the temperature on the filtered names could be fixed in advance and checked on names I have not looked at before. Third, the model could learn the legal form as a condition, so it can write names for other company types, and the excluded names with accents could be added back to reduce the language bias. Finally, a real benchmark for entity matching needs pairs with a known answer. Controlled variants of real company names, for example with changed punctuation or word order, could give such pairs as a separate test set, while the generated names would stay candidates for inputs that should not match anything. I have not shown that they are true non-matches.

References

Carlini, N., Tramèr, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, Ú., Oprea, A., & Raffel, C. (2021). Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium* (pp. 2633–2650). USENIX Association. https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting

Companies House. (n.d.). *Free Company Data Product*. Retrieved October 16, 2026, from https://download.companieshouse.gov.uk/en_output.html

Companies House. (2019). *Our public task, copyright and Crown copyright*. GOV.UK. https://www.gov.uk/government/publications/companies-house-accreditation-to-information-fair-traders-scheme/public-task-copyright-and-crown-copyright

Companies House. (2026). *Incorporation and names*. GOV.UK. https://www.gov.uk/government/publications/incorporation-and-names/incorporation-and-names

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations*. https://openreview.net/forum?id=rygGQyrFvH

Hugging Face. (n.d.). *GPT-2* [Transformers documentation, version 5.21.0]. Retrieved November 6, 2026, from https://huggingface.co/docs/transformers/v5.21.0/en/model_doc/gpt2

Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., & Carlini, N. (2022). Deduplicating training data makes language models better. In *Proceedings of the 60th*

Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 8424–8445). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.acl-long.577

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*. https://openreview.net/forum?id=Bkg6RiCqY7

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

SciPy. (n.d.). *scipy.stats.ks_2samp* [SciPy documentation]. Retrieved November 6, 2026, from https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html

Theis, L., van den Oord, A., & Bethge, M. (2016). A note on the evaluation of generative models. In *International Conference on Learning Representations*. https://arxiv.org/abs/1511.01844

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30*. Curran Associates. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html